重邮802数据结构押题卷（一）参考答案

对应临时区原《押题卷一.pdf》。按原卷题号作答；代码允许等价写法。KMP采用1基下标、next[1]=0的教材约定。题面存在笔误或条件不明时，在相应题下说明。

一、程序阅读题

1．执行次数（5分）

仅 i=1,2,...,n/2 时执行内层循环。给定 i 时，j 从2i到n，共执行 n-2i+1 次。

总次数：T(n)=Σ(i=1至n/2)(n-2i+1)=(n/2)(n+1)-2×(n/2)(n/2+1)/2=**n^2/4**。时间复杂度 O(n^2)。

2．单链表操作（5分）

（1）S1：从新的首结点起扫描，使 p 指向原链表的尾结点。

（2）S2：把原首结点 q 接到尾结点之后，再令 q->next=NULL，使 q 成为新的尾结点。

（3）n≥2 时返回的线性表为 (a2,a3,...,an,a1)，即循环左移一个位置。空表、单结点表保持原状。时间 O(n)，辅助空间 O(1)。

3．非递归折半查找（5分）

从上到下各空依次为：

```
high = n - 1;
low <= high
mid = low + (high - low) / 2;
low = mid + 1;
high = mid - 1;
```

数组须按关键字递增有序。成功返回下标，区间变空时返回-1。时间 O(log n)，辅助空间 O(1)。

4．递归折半查找（5分）

从上到下各空依次为：

```
(low <= high)
A[mid].key == K
Binsch(A, mid + 1, high, K)
return -1
```

末空后题面已有分号。原函数名 Binsh/Binsch 拼写及参数中的 high.KeyType 有笔误，统一写为同一个函数名和逗号。时间 O(log n)，递归栈辅助空间 O(log n)。

二、填空题

1．next与nextval（3分）

答案如下。next[j]等于P[1..j-1]最长相等真前后缀长度加1，首项固定0；nextval对相同字符的重复比较继续跳转。

下标	1	2	3	4	5	6	7	8	9	10	11	12
字符	c	d	d	c	d	e	c	e	c	d	e	a
next	0	1	1	1	2	3	1	2	1	2	3	1
nextval	0	1	1	0	1	3	0	2	0	1	3	1

2．多重链表空指针（3分）

(m-1)n+1，其中 n 为树的结点总数。共有 mn 个孩子指针域，其中 n-1 个对应树边，故空指针数 mn-(n-1)。原题漏给总结点数，须用 n 表示，不能仅凭度 m 得出固定数值。

3．根的左右孩子（3分）

左孩子B，右孩子F。后序末项A为根；中序切成CBED与GJIFH，对应后序CEDB与JIGHF，两部分末项分别为B、F。

4．折半比较次数（3分）

7次，2次。127=2^7-1；查K>a127沿最右路径比较a64,a96,a112,a120,a124,a126,a127；查a32依次比较a64,a32。按一次与中点关键字的三路比较计一次。

5．KMP两项（3分）

next(4)=2；nextval(7)=0。P[1..3]=cbc的最长真前后缀为c。next(7)=3，P[7]=P[3]=c，继续跳到nextval(3)=0。

6．循环队列插入位置（3分）

(rear+1) mod n。rear指实际队尾，插入到其下一槽；入队前须保证队列未满。

7．数组地址（3分）

A[2][3]。每行11个元素；50/2=25=2×11+3。

8．三叉树叶结点（3分）

21。n0=1+n2+2n3=1+6+2×7=21，度为1的结点不改变该关系。

9．拓扑排序复杂度（3分）

O(n+e)，采用邻接表和入度表实现。若采用邻接矩阵扫描，时间为O(n^2)；原题未指定存储结构，通常填前者并写明条件。

10．5阶B树最少关键字（3分）

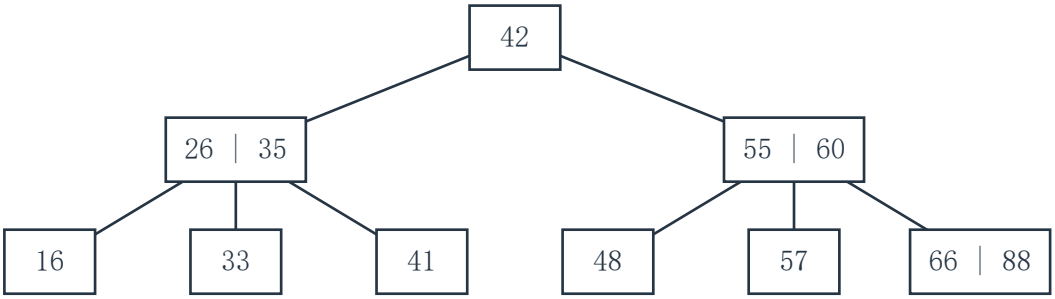
5。高度2时根至少1个关键字、2个孩子；非根结点至少ceil(5/2)-1=2个关键字，合计1+2×2=5。

三、综合应用题

1．3阶B树插入与删除（10分）

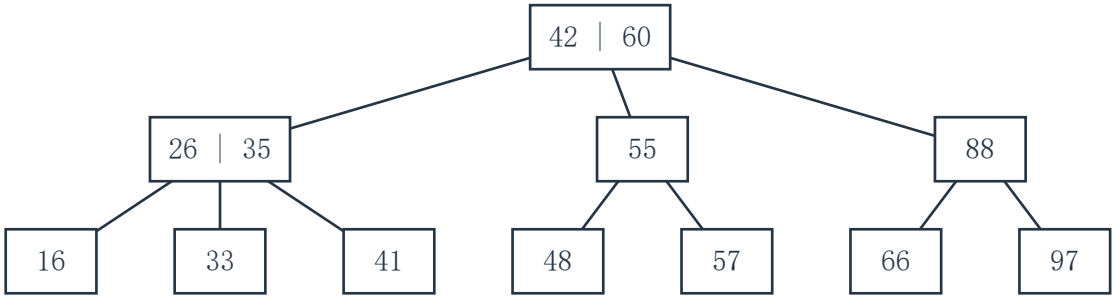
约定：每结点最多2个关键字，非根至少1个。插入溢出时中位关键字上升；本解删除内部关键字时用中序前驱替换。图中省略最底层空指针。

（1）插入33：叶[35,41]变为[33,35,41]，将35上升，左右分成[33]、[41]；其父变为[26,35]。



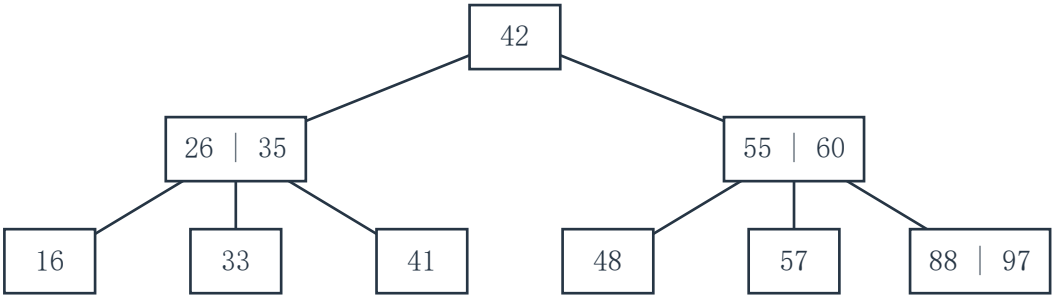
插入33后的B树

继续插入97：[66,88,97]分裂、88上升，使父[55,60,88]再次溢出，60上升到根，得到：



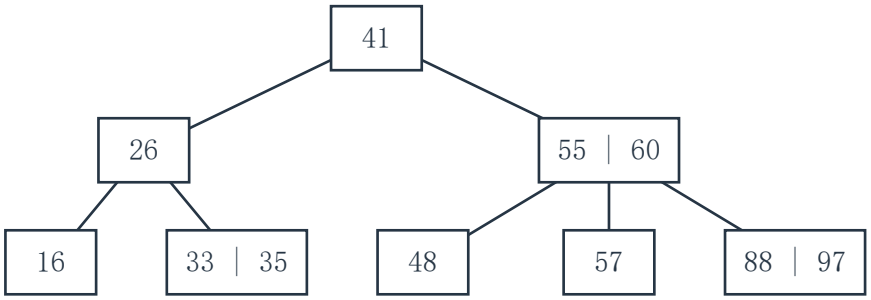
继续插入97后的B树

（2）删除66：叶[66]变空，与[97]及分隔关键字88合并成[88,97]。其父变空，再与相邻内部结点[55]及根的60合并，得到：



删除66后的B树

继续删除42：用前驱41替换根42；原叶[41]删空，与[33]及分隔关键字35合并成[33,35]，其父变为[26]。结果为：

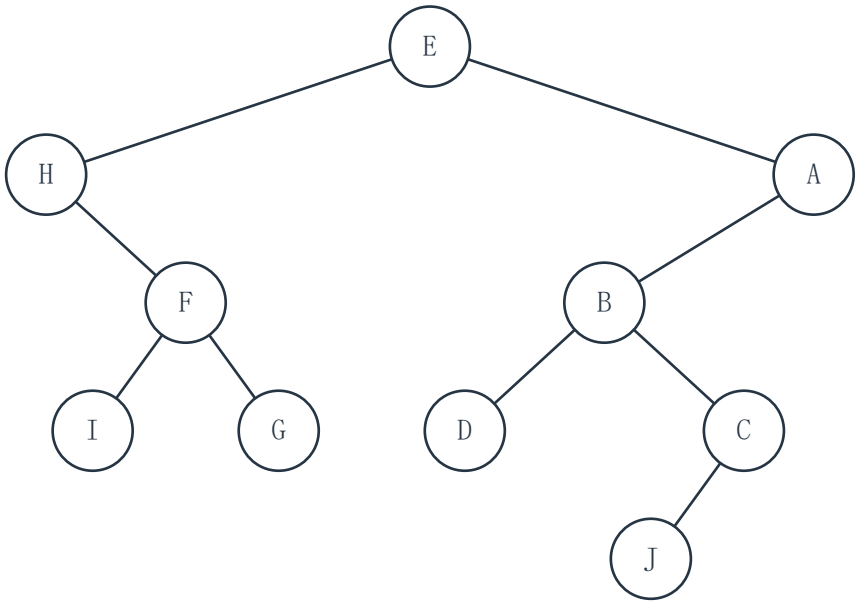


继续删除42后的B树（采用前驱41）

采用后继48替换42也可得到另一棵合法B树。只要删除后关键字集合正确、各叶同层且满足3阶B树的结点容量与查找区间规则，即为等价正确结果。

2．由先序存储表还原二叉树（10分）

题面解释：原注写Sibling为“兄弟”，但该列有2<->6、4<->5、8<->9互指，不能当作通常的右孩子链或单向右兄弟链。此处按“同父的另一个孩子下标、兄弟间双向互指，无兄弟为-1”解释；Ltag=1表示有左孩子，0表示无左孩子。按这套明示约定，原数据可自洽还原为下图。



按双向同父兄弟指针解释还原的二叉树

结点	左孩子	右孩子
E	H	A
H	空	F
F	I	G
A	B	空
B	D	C
C	J	空
I、G、D、J	空	空

先序为E,H,F,I,G,A,B,D,C,J，与原表一致；兄弟对为H<->A、I<->G、D<->C，有左孩子的结点为E、F、A、B、C，也与Ltag一致。若坚持把“右链”定义为右孩子下标，原表存在环，题面须先勘误，不能直接据此画树。

3．AOE 网络（10分）

答案：工程的最短完成时间为 23。各活动的最早开始时间与允许的最晚完成时间如下表所示。

计算过程：取拓扑序为1、4、6、5、2、3，按“前推取最大、逆推取最小”计算事件时间。

事件	最早发生时间ve的计算	最晚发生时间vl的计算
1	ve(1)=0	vl(1)=min(6-2,7-7,14-4)=0
4	ve(4)=0+2=2	vl(4)=min(7-1,15-5)=6
6	ve(6)=max(0+7,2+1)=7	vl(6)=min(15-2,14-7,23-2)=7
5	ve(5)=max(2+5,7+2)=9	vl(5)=23-8=15
2	ve(2)=max(0+4,7+7)=14	vl(2)=23-9=14
3	ve(3)=max(7+2,14+9,9+8)=23	vl(3)=23

对于活动a=(u,v)，其最早开始时间为e(a)=ve(u)，允许的最晚完成时间为LF(a)=vl(v)。若计算最晚开始时间，则l(a)=vl(v)-d(a)。本题所问为LF，不能将l直接填作最晚完成时间。

活动	弧	持续时间d	最早开始e	最晚完成LF	最晚开始l	时差l-e
a1	1→4	2	0	6	4	4
a2	1→6	7	0	7	0	0
a3	4→6	1	2	7	6	4
a4	6→5	2	7	15	13	6
a5	4→5	5	2	15	10	8
a6	1→2	4	0	14	10	10
a7	6→2	7	7	14	7	0
a8	6→3	2	7	23	21	14
a9	2→3	9	14	23	14	0
a10	5→3	8	9	23	15	6

工程最短完成时间为ve(3)=23。时差为0的关键活动是a2、a7、a9，关键路径为 **1→6→2→3**，路径长度为7+7+9=23。

4．双散列建立散列表（10分）

答案：最终散列表如下，空表示该地址尚未存放关键字。

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	55	80	35	25	70	60	45	50	空	空	空

计算过程：双重探查的第i次地址为H_i(key)=[h1(key)+i×h2(key)] mod11，其中i从0开始，h1(key)=key mod11，h2(key)=key mod9+1。

插入关键字	h1	h2	按顺序探测的地址	最终地址
70	4	8	4	4
25	3	8	3	3
80	3	9	3→1	1
35	2	9	2	2
60	5	7	5	5
45	1	1	1→2→3→4→5→6	6
50	6	6	6→1→7	7
55	0	2	0	0

例如，插入50时，初始地址为6，步长为6。地址6已被45占用，再探测(6+6) mod11=1，仍有80，再探测(6+2×6) mod11=7，故将50存入地址7。

5．堆排序过程（10分）

答案：采用大根堆进行升序堆排序，最终序列为 **2，28，52，54，71，72，74，93**。

建堆过程：将初始序列看作完全二叉树，按1起始下标，从最后一个非叶结点i=4向根依次向下筛选。

步骤	调整结果（顺序存储序列）
初始序列	71，74，2，72，54，93，52，28
调整i=4	71，74，2，72，54，93，52，28
调整i=3	71，74，93，72，54，2，52，28
调整i=2	71，74，93，72，54，2，52，28
调整i=1，建堆完成	93，74，71，72，54，2，52，28

排序过程：每趟将堆顶与当前堆末元素交换，缩小堆的范围，再将新堆顶向下筛选。下表给出每趟筛选完成后的状态，右侧为已排好序的区间。

趟数	剩余大根堆	已排好序的区间
1	74，72，71，28，54，2，52	93
2	72，54，71，28，52，2	74，93
3	71，54，2，28，52	72，74，93
4	54，52，2，28	71，72，74，93
5	52，28，2	54，71，72，74，93
6	28，2	52，54，71，72，74，93
7	2	28，52，54，71，72，74，93

6．不能得到的出栈序列（10分）

唯一不可能的序列：CAB。C首先出栈时，A、B、C必已全部入栈；弹出C后，栈顶为B，必须先弹B再弹A，故不能接着弹A。

其余五种排列ABC、ACB、BAC、BCA、CBA均可由合法入出栈操作得到。

7．顺序表插入的平均移动次数（10分）

长为n的顺序表有n+1个插入位置：第一个元素之前，以及每个元素之后。各位置等概率1/(n+1)。

在第i个元素之前插入（i=1,...,n+1）时，需后移n-i+1个元素；i=n+1即表尾插入，移动0个。

平均移动次数：E=Σ(i=1至n+1)[(n-i+1)/(n+1)]=(n+(n-1)+...+1+0)/(n+1)=**n/2**。

时间复杂度平均O(n)，辅助空间O(1)；这里统计元素后移次数，不把写入新元素本身算作移动。

四、算法设计与分析题

1．Floyd 各步矩阵与路径输出（10分）

（1）答案与计算过程

行、列均按顶点1、2、3、4排列；a[i][i]=0，无直接弧记为∞。path[i][j]=0表示该路径没有中转顶点；只有当a[i][k]+a[k][j]严格小于当前a[i][j]时，才更新距离并令path[i][j]=k。等长时保留原记录。

初始矩阵如下。

```
a(0) =                    path(0) =
[ 0   1   ∞   4 ]          [ 0  0  0  0 ]
[ ∞   0   9   2 ]          [ 0  0  0  0 ]
[ 3   5   0   8 ]          [ 0  0  0  0 ]
[ ∞   ∞   6   0 ]            [ 0  0  0  0 ]
```

k=1时，a[3][2]由5降为3+1=4，a[3][4]由8降为3+4=7。

```
a(1) =                    path(1) =
[ 0   1   ∞   4 ]          [ 0  0  0  0 ]
[ ∞   0   9   2 ]          [ 0  0  0  0 ]
[ 3   4   0   7 ]          [ 0  1  0  1 ]
[ ∞   ∞   6   0 ]            [ 0  0  0  0 ]
```

k=2时，a[1][3]更新为10，a[1][4]更新为3，a[3][4]更新为6。

```
a(2) =                    path(2) =
[ 0   1  10   3 ]          [ 0  0  2  2 ]
[ ∞   0   9   2 ]          [ 0  0  0  0 ]
[ 3   4   0   6 ]          [ 0  1  0  2 ]
[ ∞   ∞   6   0 ]            [ 0  0  0  0 ]
```

k=3时，a[2][1]更新为12，a[4][1]更新为9，a[4][2]更新为10。

```
a(3) =                    path(3) =
[ 0   1  10   3 ]          [ 0  0  2  2 ]
[12   0   9   2 ]          [ 3  0  0  0 ]
[ 3   4   0   6 ]          [ 0  1  0  2 ]
[ 9  10   6   0 ]          [ 3  3  0  0 ]
```

k=4时，a[1][3]由10降为9，a[2][1]由12降为11，a[2][3]由9降为8。最终矩阵如下。

```
a(4) =                    path(4) =
[ 0   1   9   3 ]          [ 0  0  4  2 ]
[11   0   8   2 ]          [ 4  0  4  0 ]
[ 3   4   0   6 ]          [ 0  1  0  2 ]
[ 9  10   6   0 ]          [ 3  3  0  0 ]
```

最终各对不同顶点间的最短路径如下。

顶点对	最短路径	长度
1→2	1→2	1
1→3	1→2→4→3	9
1→4	1→2→4	3
2→1	2→4→3→1	11
2→3	2→4→3	8

顶点对	最短路径	长度
2→4	2→4	2
3→1	3→1	3
3→2	3→1→2	4
3→4	3→1→2→4	6
4→1	4→3→1	9
4→2	4→3→1→2	10
4→3	4→3	6

（2）路径输出算法

基本思想：若path[i][j]=k≠0，则i到j的路径分为i到k和k到j两段。递归输出前一段的内部中转顶点，再输出k，最后输出后一段的内部中转顶点。最外层输出起点i和终点j，避免重复输出分段端点。不可达时单独报告无路径；i=j时只输出该顶点一次，长度为0。

完整C实现：调用前将a初始化为带权邻接矩阵，对角线为0，无弧为INF。先调用Floyd(n,a,path)，再调用PrintAllPaths(n,a,path)。若题目已经给出Floyd计算结果，可直接调用PrintAllPaths。

```
#include <stdio.h>
#include <limits.h>

#define MAXV 100
#define INF (LLONG_MAX / 4)

/* 顶点下标为1..n；path=0表示没有内部中转顶点。 */
void Floyd(int n, long long a[MAXV + 1][MAXV + 1],
           int path[MAXV + 1][MAXV + 1])
{
    int i, j, k;
    for (i = 1; i <= n; ++i)
        for (j = 1; j <= n; ++j)
            path[i][j] = 0;

    for (k = 1; k <= n; ++k) {
        for (i = 1; i <= n; ++i) {
            for (j = 1; j <= n; ++j) {
                if (a[i][k] == INF || a[k][j] == INF)
                    continue;
                long long cand = a[i][k] + a[k][j];
                if (cand < a[i][j]) {
                    a[i][j] = cand;
                    path[i][j] = k;
                }
            }
        }
    }
}

/* 只输出i到j之间的内部中转顶点。 */
void PrintInner(int i, int j,
                int path[MAXV + 1][MAXV + 1])
{
    int k = path[i][j];
    if (k == 0)
        return;
    PrintInner(i, k, path);
```

```
    printf(" -> %d", k);
    PrintInner(k, j, path);
}

void PrintAllPaths(int n,
                   long long a[MAXV + 1][MAXV + 1],
                   int path[MAXV + 1][MAXV + 1])
{
    int i, j;
    for (i = 1; i <= n; ++i) {
        for (j = 1; j <= n; ++j) {
            printf("<%d,%d>: ", i, j);
            if (a[i][j] == INF) {
                printf("unreachable\n");
                continue;
            }
            printf("length=%lld, path=%d", a[i][j], i);
            if (i != j) {
                PrintInner(i, j, path);
                printf(" -> %d", j);
            }
            printf("\n");
        }
    }
}
```

复杂度：Floyd三重循环的时间复杂度为O(n^3)，距离矩阵与中转矩阵占O(n^2)空间。对已求出的矩阵输出全部路径，时间为O(n^2+所有输出路径的顶点总数)，最坏为O(n^3)；递归输出一条简单路径的辅助栈空间最坏为O(n)。

2．循环链表中的括号配对（10分）

基本思想：从L->link起扫描，回到表头L时结束。遇左括号入栈，遇右括号时要求栈非空且栈顶与其匹配，然后出栈。其他字符略过。扫描后栈空才匹配。以下同时支持圆、方、花括号。

C语言实现：

```
#include <stdlib.h>
typedef struct Node {
    char ch;
    struct Node *link;
} Node;
static int pair(char a, char b) {
    return (a == '(' && b == ')') ||
           (a == '[' && b == ']') ||
           (a == '{' && b == '}');
}
int bracketsMatch(const Node *L) {
    int n = 0, top = 0;
    const Node *p;
    for (p = L->link; p != L; p = p->link) ++n;
    char *st = malloc((size_t)n + 1);
    if (st == NULL) abort();
    for (p = L->link; p != L; p = p->link) {
        char c = p->ch;
        if (c == '(' || c == '[' || c == '{') st[top++] = c;
        else if (c == ')' || c == ']' || c == '}') {
            if (top == 0 || !pair(st[top - 1], c)) {
                free(st);
                return 0;
            }
            --top;
        }
    }
    int ok = (top == 0);
    free(st);
    return ok;
}
```

复杂度：设表达式含n个字符，两次扫描均为O(n)，栈最多存n个字符，辅助空间O(n)。空表达式返回1。只要求圆括号时也可用计数器，遇右括号先判计数是否为0，最终判0，辅助空间O(1)。

3．带子树规模的BST中第k小结点（原卷本题另标16分）

基本思想：设左子树结点数为s，则当前根的排名是s+1。k=s+1返回根；k≤s向左递归；否则向右递归查第k-s-1小。每次仅进入一个子树，直接读取num即可取得左子树规模。

C语言实现：

```
#include <stddef.h>
typedef struct BSTNode {
    int element, num;
    struct BSTNode *lchild, *rchild;
} BSTNode;
BSTNode *selectK(BSTNode *T, int k) {
    if (T == NULL || k < 1 || k > T->num) return NULL;
    int s = T->lchild ? T->lchild->num : 0;
    if (k == s + 1) return T;
    if (k <= s) return selectK(T->lchild, k);
    return selectK(T->rchild, k - s - 1);
}
```

复杂度与理由：每层常数次操作，时间O(h)，递归栈辅助空间O(h)。按互异关键字随机排列、依次插入建立BST的通常随机模型，平均查找路径长度为O(log n)，故平均时间、平均栈空间均为O(log n)。最坏退化为链时二者均为O(n)。原题rechild按rchild解释，num已正确预存，无须在查询中重新统计。

题面分值说明：第四大题标题写3题各10分，本题末尾又标16分，二者冲突。本答案保留题号并完整解答，不自行重分总分。